Networks with Adversarial Examples

I. Attacking NNs with Adversarial Examples

II. Generative Adversarial Networks

I. Adversarial examples

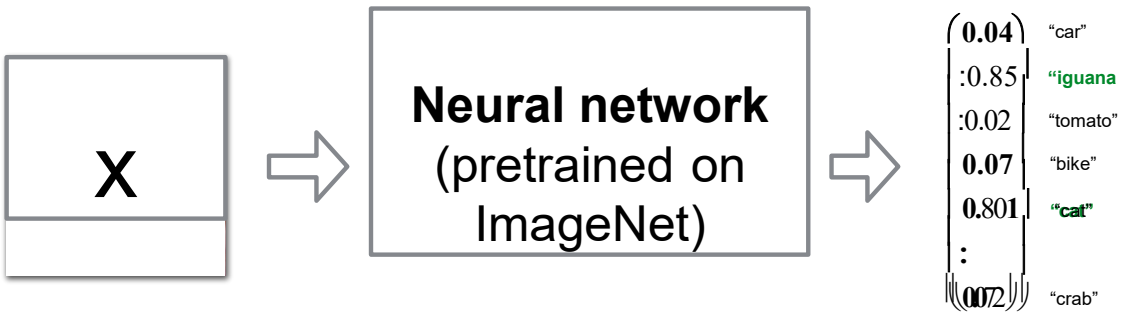
A. Attacking a network with adversarial examples

B. Defenses against adversarial examples

C. Why are neural networks vulnerable to adversarial examples?

[Szegedy et al. (2013): Intriguing properties of neural networks]
[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

Goal: Given a network pretrained on ImageNet, find an input image that is not an iguana but will be classified as an iguana.



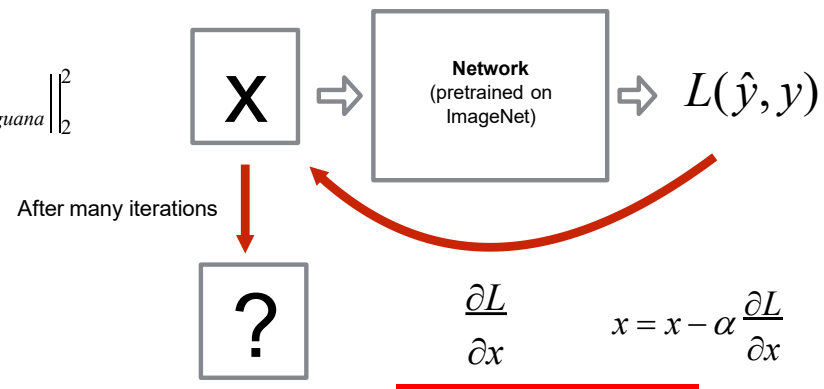
1. Rephrasing what we want:

Find x such that: $\hat{y}(x) = y_{iguana} = \begin{pmatrix} 0 \\ 1 \\ : \\ 0 \\ 0 \\ : \\ 0 \end{pmatrix}$

2. Defining the loss function

$$L(\hat{y}, y) = \frac{1}{2}\left\|\hat{y}(W, b, x) - y_{iguana}\right\|_2^2$$

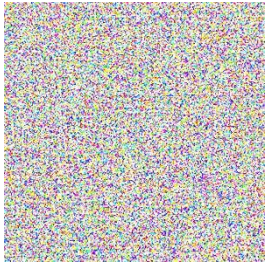
3. Optimize the image



[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

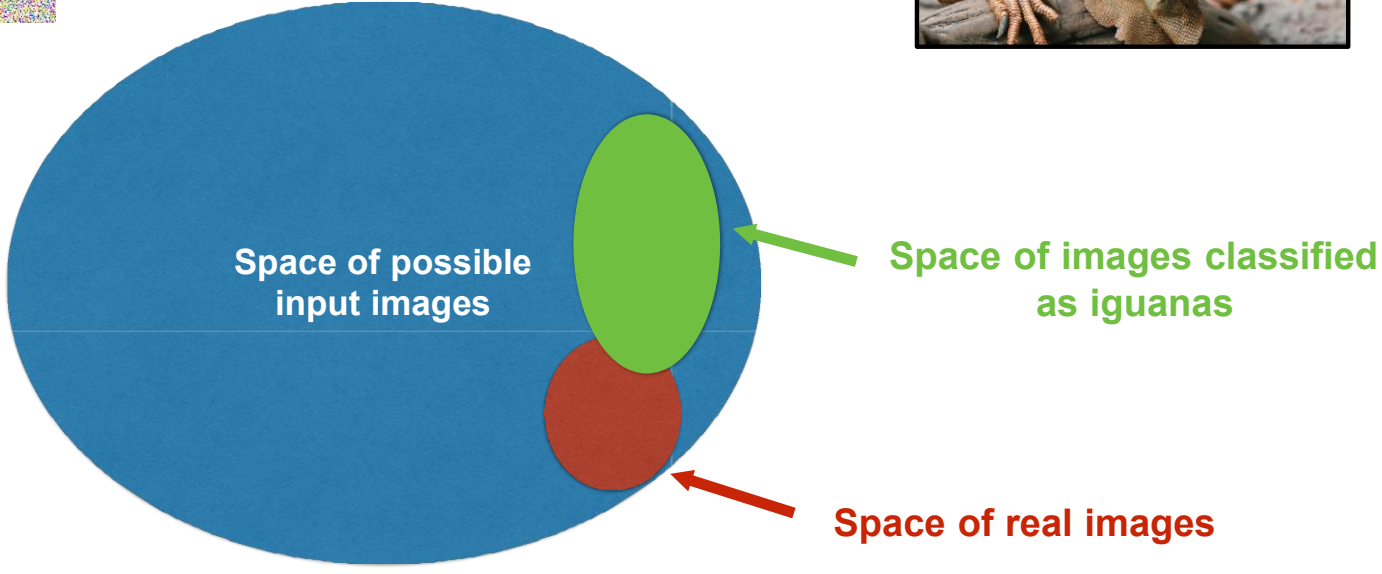
I. A. Attacking a network with adversarial examples

Question: Will the forged image **x** look like an **iguana**?



?

$$256^{32\times32\times3} \approx 10^{7400}$$



Goal: Given a network pretrained on ImageNet, find an input image that is a cat but will be classify as an iguana.

Neural network (pretrained on ImageNet)

$$\begin{pmatrix} 0.04 \\ :0.85 \\ :0.02 \\ 0.07 \\ 0.801 \\ : \\ 0.02 \end{pmatrix}$$

"car"
"iguana"
"tomato"
"bike"
"cat"
"crab"

1. Rephrasing what we want:

Find x such that: $\hat{y}(x) = y_{iguana} = \begin{pmatrix} 0 \\ 1 \\ : \\ 0 \\ 0 \\ : \\ 0 \end{pmatrix}$

And: $x = x_{cat}$

2. Defining the loss function

$$L(\hat{y}, y) = \frac{1}{2}\left\|\hat{y}(W, b, x) - y_{iguana}\right\|_2^2 + \lambda\left\|x - x_{cat}\right\|_2^2$$

After many iterations

3. Optimize the image

Network (pretrained on ImageNet)

$L(\hat{y}, y)$

$$\frac{\partial L}{\partial x}$$

$$x = x - \alpha \frac{\partial L}{\partial x}$$

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

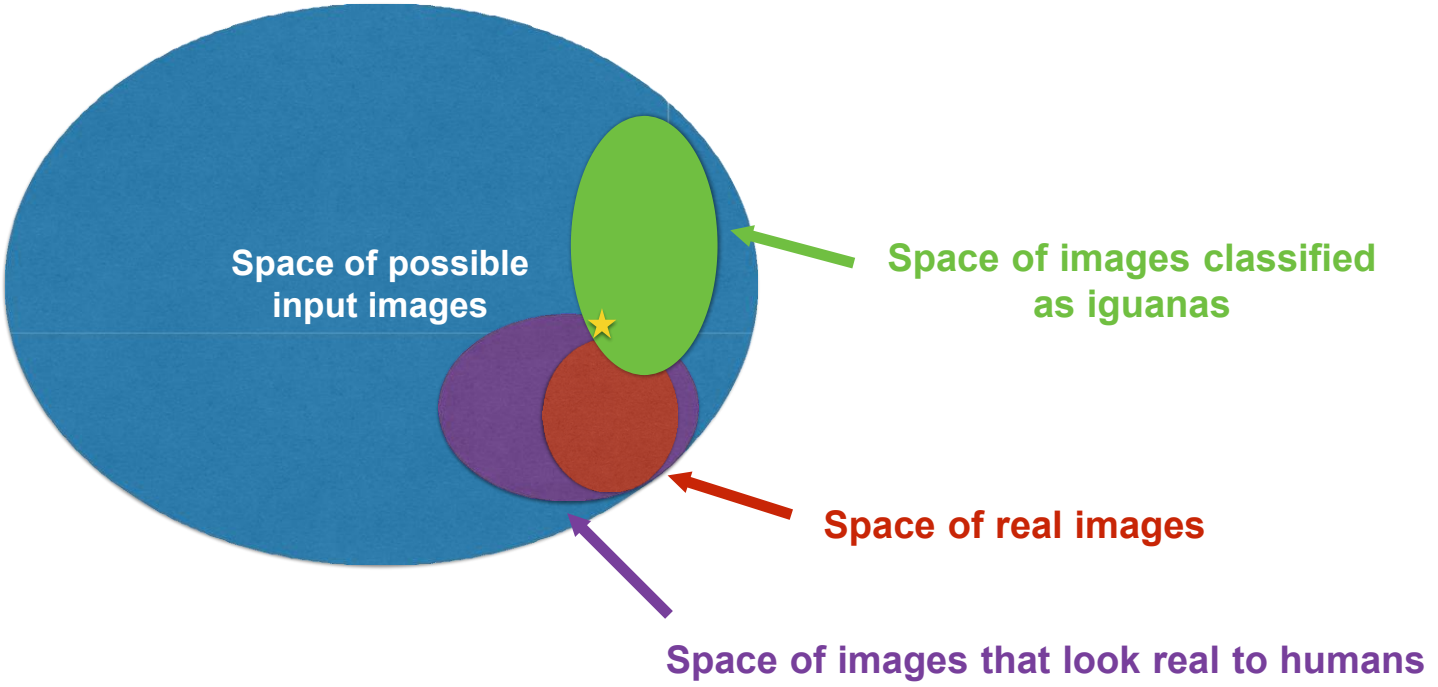
I. A. Attacking a network with adversarial examples

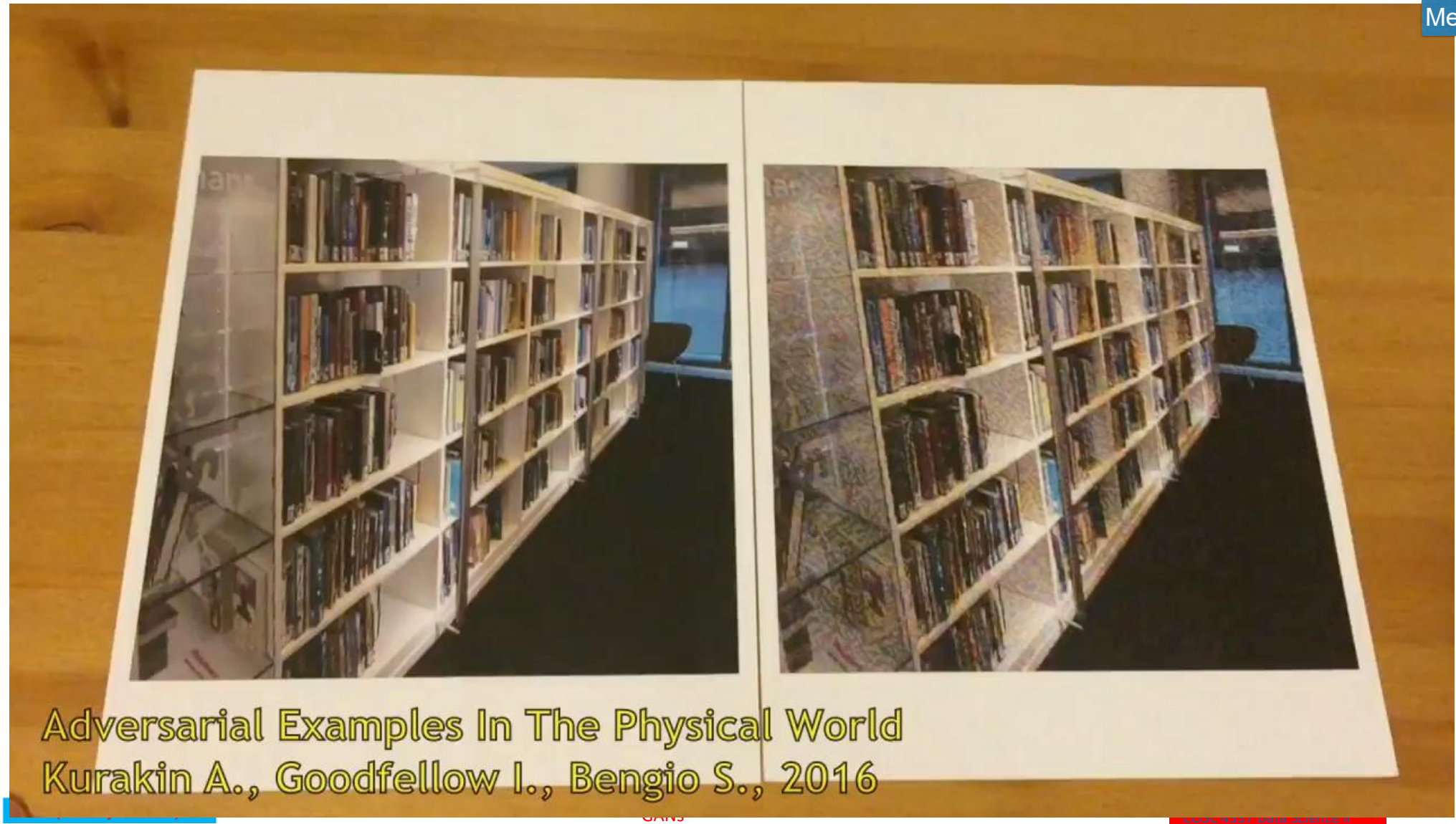
92% Cat

94% Iguana

I. A. Attacking a network with adversarial examples

$$256^{32\times32\times3} \approx 10^{7400}$$





[Alexey Kurakin, Ian J. Goodfellow, Samy Bengio (2017): Adversarial examples in the physical world]

I. B. Defenses against adversarial examples

Menti

Knowledge of the attacker:

- White-box
- Black-box

Solution 1

- Create a SafetyNet

x = 

y = cat

Solution 2

- Train on correctly labelled adversarial examples

Solution 3

- Adversarial training $L_{new} = L(W,b,x,y) + \lambda L(W,b,x_{adv},y)$
- Adversarial logit pairing $L_{new} = L(W,b,x,y) + \lambda \left|\left| f(x;W,b) - f(x_{adv};W,b) \right|\right|_2^2$

[Lu et al. (2017): SafetyNet: Detecting and Rejecting Adversarial Examples Robustly]
[Harini Kannan et al. (2018): Adversarial Logit Pairing]

II. Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs

D. In terms of code

E. Nice results

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

II.A - Motivation

Motivation: endowing computers with an understanding of our world.

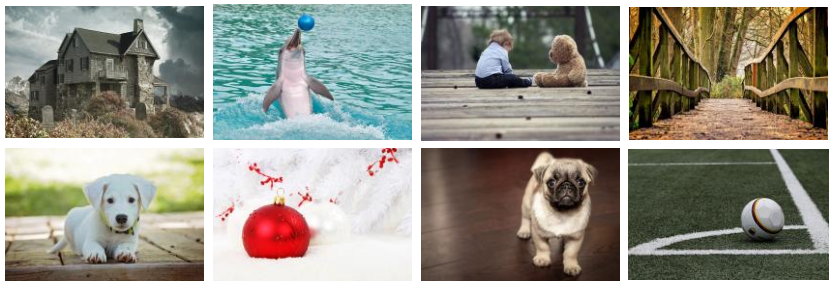
Goal: collect a lot of data, use it to train a model to generate similar data from scratch.

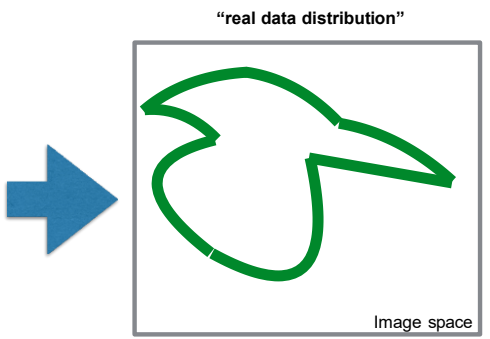
Intuition: number of parameters of the model << amount of data

II.A - Motivation

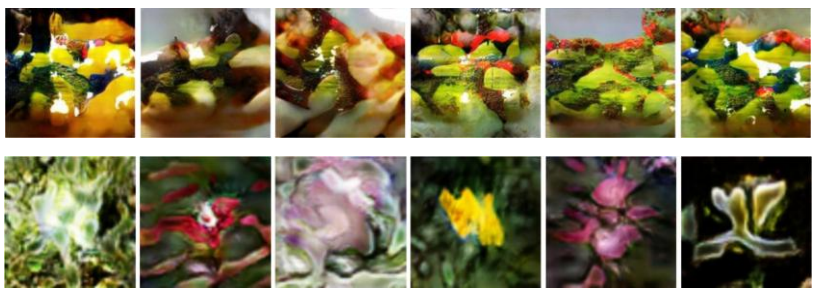
Probability distributions:

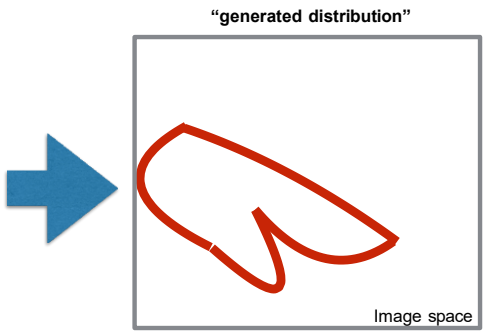
Samples from the “real data distribution”



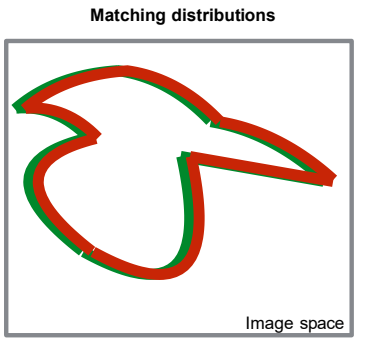


Samples from the “generated distribution”





Goal



[Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, Dimitris Metaxas (2017): StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks]

II. Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs

D. In terms of code

E. Nice results

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

II.B - G/D Game

How can we train G to generate images from the true data distributions?

[Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, Dimitris Metaxas (2017): StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks]

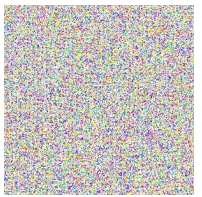
II.B - G/D Game

100-d
random code

$$z = \begin{pmatrix} 0.47 \\ \vdots \\ 0.19 \end{pmatrix}$$

Generator "G"
(Neural Network)

(64,64,3)
generated image



Run Adam simultaneously on two minibatches
(true data / generated data)

Gradients

Binary classification

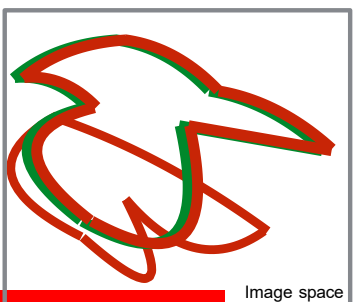
$$\begin{cases} y = 0 \quad if \quad x = G(z) \\ y = 1 \quad otherwise \end{cases}$$

x

Discriminator "D"
(Neural Network)

Real images
(database)

Probability distributions



II.B - G/D Game

100-d
random code

(64,64,3)
generated image

$$\begin{pmatrix} 0.47 \\ \vdots \\ 0.19 \end{pmatrix}$$

z

Generator "G" (Neural Network)

End goal: G is outputting images that are indistinguishable from real images for D

x

Discriminator "D" (Neural Network)

Gradients

Binary classification

$$\begin{cases} y = 0 \quad if \quad x = G(z) \\ y = 1 \quad otherwise \end{cases}$$

Real images (database)

Probability distribution

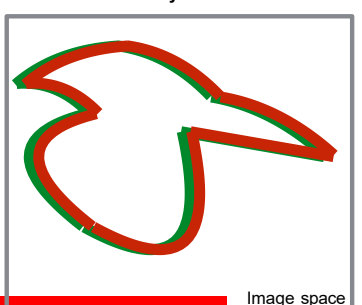


Image space

II.B - G/D Game

Training procedure, we want to minimize:

Labels: $\begin{cases} y_{real} \text{ is always 1} \\ y_{gen} \text{ is always 0} \end{cases}$

- The cost of the discriminator

$$J^{(D)} = \underbrace{-\frac{1}{m_{real}} \sum_{i=1}^{m_{real}} y_{real}^{(i)} . \log(D(x^{(i)}))}_{\text{cross-entropy 1}} \underbrace{-\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} (1 - y_{gen}^{(i)}) . \log(1 - D(G(z^{(i)})))}_{\text{cross-entropy 2}}$$

cross-entropy 1:
"D should correctly label real data as 1"

cross-entropy 2:
"D should correctly label generated data as 0"

- The cost of the generator

$$J^{(G)} = -J^{(D)} = \frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)})))$$

"G should try to fool D: by minimizing the opposite of what D is trying to minimize"

II. Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs

D. In terms of code

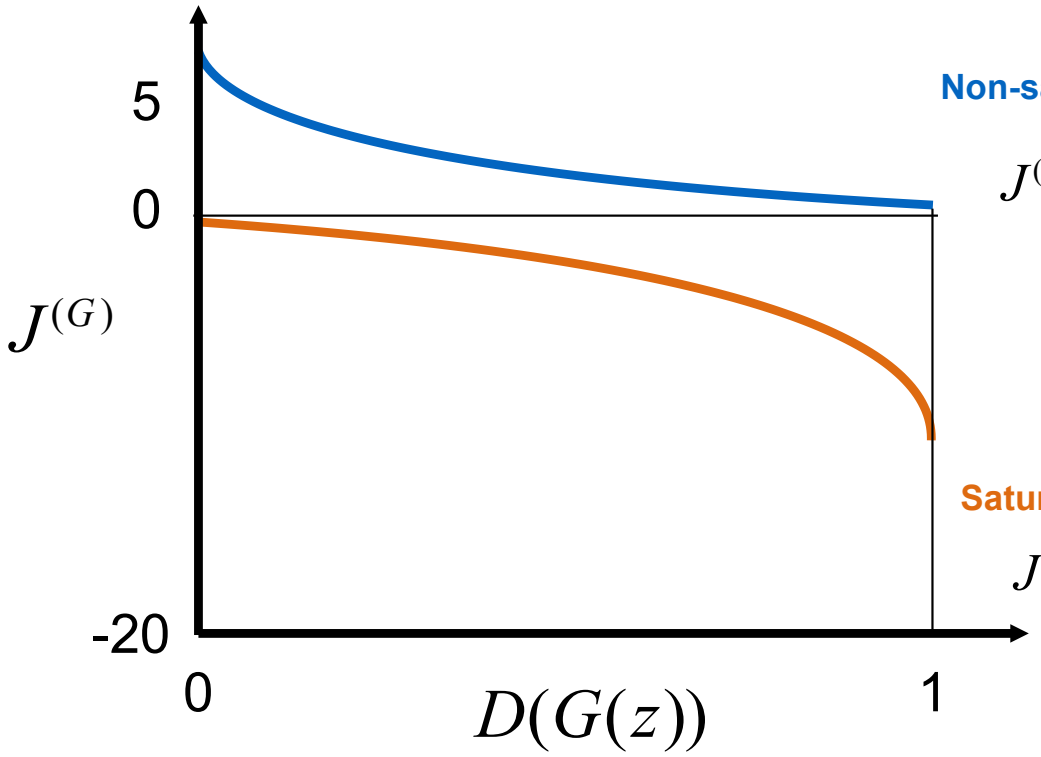
E. Nice results

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

II.C - Training GANs

Saturating cost for the generator:

$$\min\left[\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(1-D(G(z^{(i)})))\right] \Leftrightarrow \max\left[\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))\right] \Leftrightarrow \min\left[-\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))\right]$$



Non-saturating cost

$$J^{(G)} = -\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))$$

Saturating cost

$$J^{(G)} = \frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(1-D(G(z^{(i)})))$$

[Ian Goodfellow (2014): NIPS Tutorial: GANs]

II.C - Training GANs

Note that: $$\min\left[\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(1-D(G(z^{(i)})))\right] \Leftrightarrow \max\left[\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))\right] \Leftrightarrow \min\left[-\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))\right]$$

New training procedure, we want to minimize:

$$J^{(D)} = -\frac{1}{m_{real}}\sum_{i=1}^{m_{real}} y_{real}^{(i)}.\log(D(x^{(i)})) - \frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}(1-y_{gen}^{(i)}).\log(1-D(G(z^{(i)})))$$

cross-entropy 1:
"D should correctly label real data as 1"

cross-entropy 2:
"D should correctly label generated data as 0"

$$J^{(G)} = -\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D(G(z^{(i)})))$$

"G should try to fool D: by minimizing this"

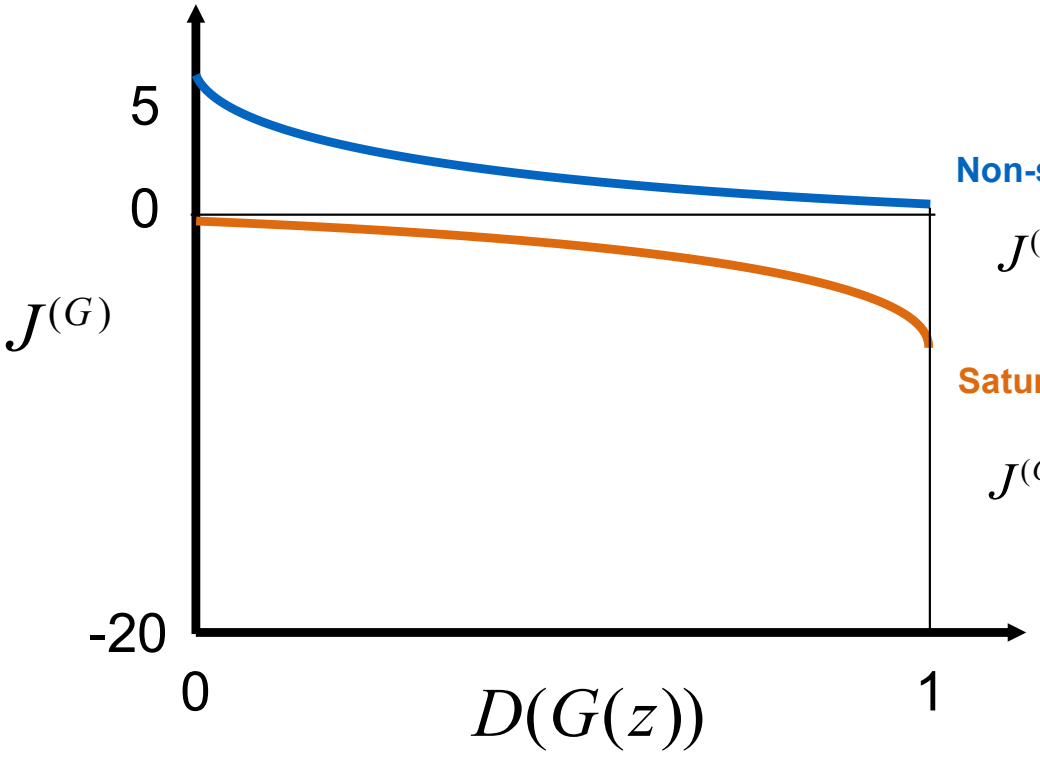
Table 1: Generator and discriminator loss functions. The main difference whether the discriminator outputs a probability (MM GAN, NS GAN, DRAGAN) or its output is unbounded (WGAN, WGAN GP, LS GAN, BEGAN), whether the gradient penalty is present (WGAN GP, DRAGAN) and where is it evaluated. We chose those models based on their popularity.

GAN	Discriminator Loss	Generator Loss
MM GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{GAN}} = -\mathbb{E}_{x\sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x}\sim p_g}[\log(1-D(\hat{x}))]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{GAN}} = \mathbb{E}_{\hat{x}\sim p_g}[\log(1-D(\hat{x}))]$
NS GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{NSGAN}} = -\mathbb{E}_{x\sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x}\sim p_g}[\log(1-D(\hat{x}))]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{NSGAN}} = -\mathbb{E}_{\hat{x}\sim p_g}[\log(D(\hat{x}))]$
WGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{WGAN}} = -\mathbb{E}_{x\sim p_d}[D(x)] + \mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{WGAN}} = -\mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$
WGAN GP	$\mathcal{L}_{\mathrm{D}}^{\mathrm{WGANGP}} = \mathcal{L}_{\mathrm{D}}^{\mathrm{WGAN}} + \lambda\mathbb{E}_{\hat{x}\sim p_g}[(\|\nabla D(\alpha x + (1-\alpha\hat{x})\|_2 - 1)^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{WGANGP}} = -\mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})]$
LS GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{LSGAN}} = -\mathbb{E}_{x\sim p_d}[(D(x)-1)^2] + \mathbb{E}_{\hat{x}\sim p_g}[D(\hat{x})^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{LSGAN}} = -\mathbb{E}_{\hat{x}\sim p_g}[(D(\hat{x}-1)^2]$
DRAGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{DRAGAN}} = \mathcal{L}_{\mathrm{D}}^{\mathrm{GAN}} + \lambda\mathbb{E}_{\hat{x}\sim p_d+\mathcal{N}(0,c)}[(\|\nabla D(\hat{x})\|_2 - 1)^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{DRAGAN}} = \mathbb{E}_{\hat{x}\sim p_g}[\log(1-D(\hat{x}))]$
BEGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{BEGAN}} = \mathbb{E}_{x\sim p_d}[\|x - \mathrm{AE}(x)\|_1] - k_t\mathbb{E}_{\hat{x}\sim p_g}[\|\hat{x} - \mathrm{AE}(\hat{x})\|_1]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{BEGAN}} = \mathbb{E}_{\hat{x}\sim p_g}[\|\hat{x} - \mathrm{AE}(\hat{x})\|_1]$

[Lucic, Kurach et al. (2018): Are GANs Created Equal? A Large-Scale Study]

Simultaneously training G/D?

```
for num_iterations:

    for k iterations:
        update D

    update G
```



Non-saturating cost

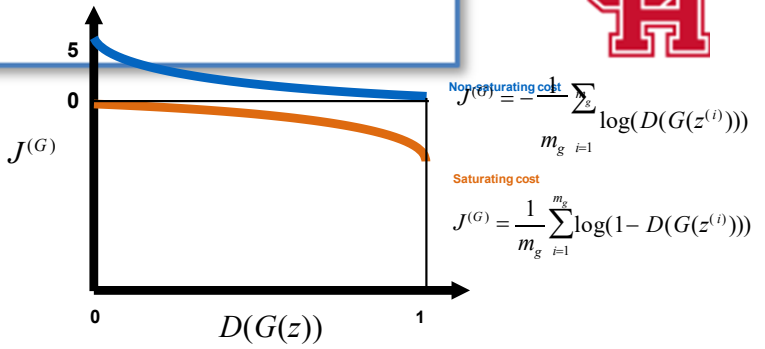
$$J^{(G)} = -\frac{1}{m_g}\sum_{i=1}^{m_g}\log(D(G(z^{(i)})))$$

Saturating cost

$$J^{(G)} = \frac{1}{m_g}\sum_{i=1}^{m_g}\log(1-D(G(z^{(i)})))$$

[Ian Goodfellow (2014): NIPS Tutorial: GANs]

II.C - Training GANs



Recap: GANs' training tips

- Modification of the cost function
- Keep D up-to-date with respect to G (k update for D / 1 update for G)
- (not presented but important) Use Virtual Batchnorm

And a lot more, GANs are hard to train!

[Soumith et al. (2016): GanHacks]
[Lucic, Kurach et al. (2018): Are GANs Created Equal? A Large-Scale Study]

II. Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs

D. In terms of code

E. Nice results

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

II. D. In terms of code

```
# Build and compile the discriminator
self.discriminator = self.build_discriminator()
self.discriminator.compile(loss='binary_crossentropy',
    optimizer=optimizer,
    metrics=['accuracy'])

# Build the generator
self.generator = self.build_generator()

# The generator takes noise as input and generates imgs
z = Input(shape=(self.latent_dim,))
img = self.generator(z)

# For the combined model we will only train the generator
self.discriminator.trainable = False

# The discriminator takes generated images as input and determines validity
validity = self.discriminator(img)

# The combined model  (stacked generator and discriminator)
# Trains the generator to fool the discriminator
self.combined = Model(z, validity)
self.combined.compile(loss='binary_crossentropy', optimizer=optimizer)
```

```
def build_discriminator(self):

    model = Sequential()

    model.add(Flatten(input_shape=self.img_shape))
    model.add(Dense(512))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dense(256))
    model.add(LeakyReLU(alpha=0.2))
    model.add(Dense(1, activation='sigmoid'))
    model.summary()

    img = Input(shape=self.img_shape)
    validity = model(img)

    return Model(img, validity)
```

[Erik Linder-Norén (Github): eriklindernoren/Keras-GAN: link]

II. D. In terms of code

```
# Build and compile the discriminator
self.discriminator = self.build_discriminator()
self.discriminator.compile(loss='binary_crossentropy',
    optimizer=optimizer,
    metrics=['accuracy'])

# Build the generator
self.generator = self.build_generator()

# The generator takes noise as input and generates imgs
z = Input(shape=(self.latent_dim,))
img = self.generator(z)

# For the combined model we will only train the generator
self.discriminator.trainable = False

# The discriminator takes generated images as input and determines validity
validity = self.discriminator(img)

# The combined model  (stacked generator and discriminator)
# Trains the generator to fool the discriminator
self.combined = Model(z, validity)
self.combined.compile(loss='binary_crossentropy', optimizer=optimizer)
```

```
def build_generator(self):

    model = Sequential()

    model.add(Dense(256, input_dim=self.latent_dim))
    model.add(LeakyReLU(alpha=0.2))
    model.add(BatchNormalization(momentum=0.8))
    model.add(Dense(512))
    model.add(LeakyReLU(alpha=0.2))
    model.add(BatchNormalization(momentum=0.8))
    model.add(Dense(1024))
    model.add(LeakyReLU(alpha=0.2))
    model.add(BatchNormalization(momentum=0.8))
    model.add(Dense(np.prod(self.img_shape), activation='tanh'))
    model.add(Reshape(self.img_shape))

    model.summary()

    noise = Input(shape=(self.latent_dim,))
    img = model(noise)

    return Model(noise, img)
```

[Erik Linder-Norén (Github): eriklindernoren/Keras-GAN: link]

II. D. In terms of code

```
92      def train(self, epochs, batch_size=128, sample_interval=50):
93
94          # Load the dataset
95          (X_train, _), (_, _) = mnist.load_data()
96
97          # Rescale -1 to 1
98          X_train = X_train / 127.5 - 1.
99          X_train = np.expand_dims(X_train, axis=3)
100
101         # Adversarial ground truths
102         valid = np.ones((batch_size, 1))
103         fake = np.zeros((batch_size, 1))
104
```

```
105         for epoch in range(epochs):
106
107             # ---------------------
108             #  Train Discriminator
109             # ---------------------
110
111             # Select a random batch of images
112             idx = np.random.randint(0, X_train.shape[0], batch_size)
113             imgs = X_train[idx]
114
115             noise = np.random.normal(0, 1, (batch_size, self.latent_dim))
116
117             # Generate a batch of new images
118             gen_imgs = self.generator.predict(noise)
119
120             # Train the discriminator
121             d_loss_real = self.discriminator.train_on_batch(imgs, valid)
122             d_loss_fake = self.discriminator.train_on_batch(gen_imgs, fake)
123             d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
124
125             # ---------------------
126             #  Train Generator
127             # ---------------------
128
129             noise = np.random.normal(0, 1, (batch_size, self.latent_dim))
130
131             # Train the generator (to have the discriminator label samples as valid)
132             g_loss = self.combined.train_on_batch(noise, valid)
```

[Erik Linder-Norén (Github): eriklindernoren/Keras-GAN: link]

II. Generative Adversarial Networks (GANs)

A. Motivation

B. G/D Game

C. Training GANs

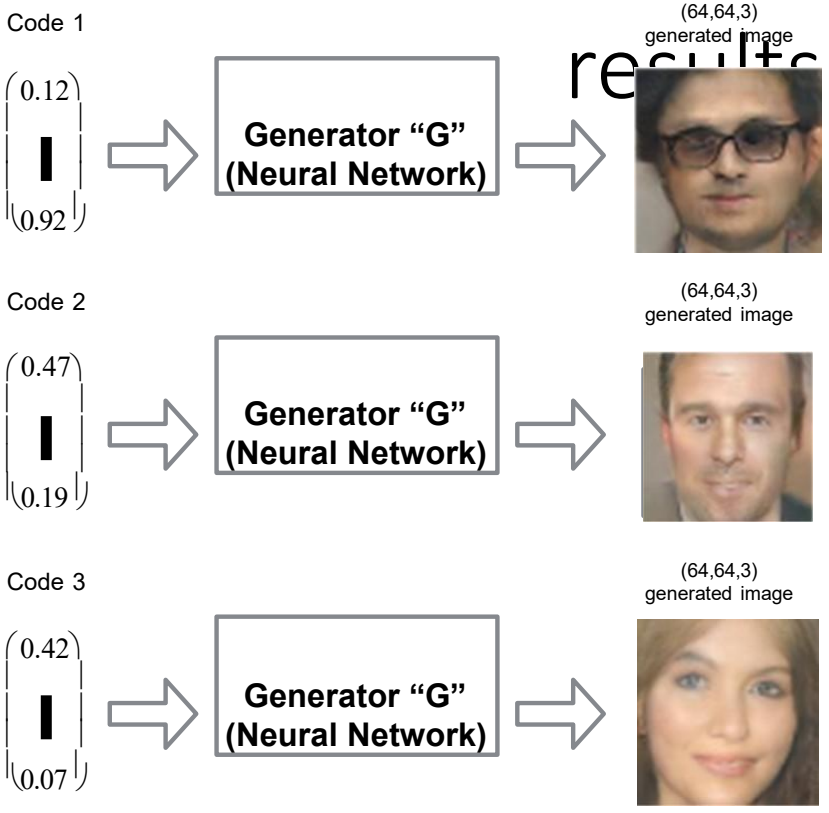
D. In terms of code

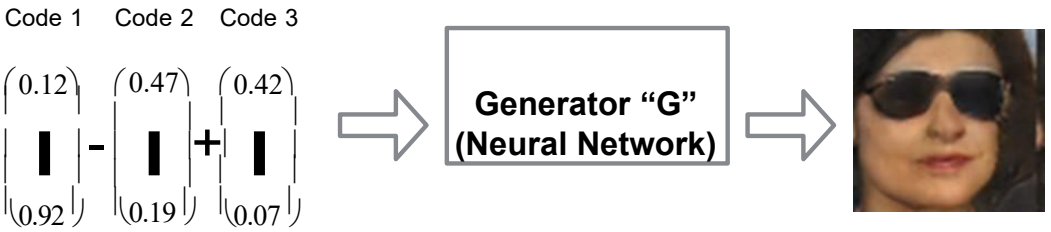
E. Nice results

[Ian J. Goodfellow, Jonathon Shlens & Christian Szegedy (2015): Explaining and harnessing adversarial examples]

II.E - Nice results

Operation on codes



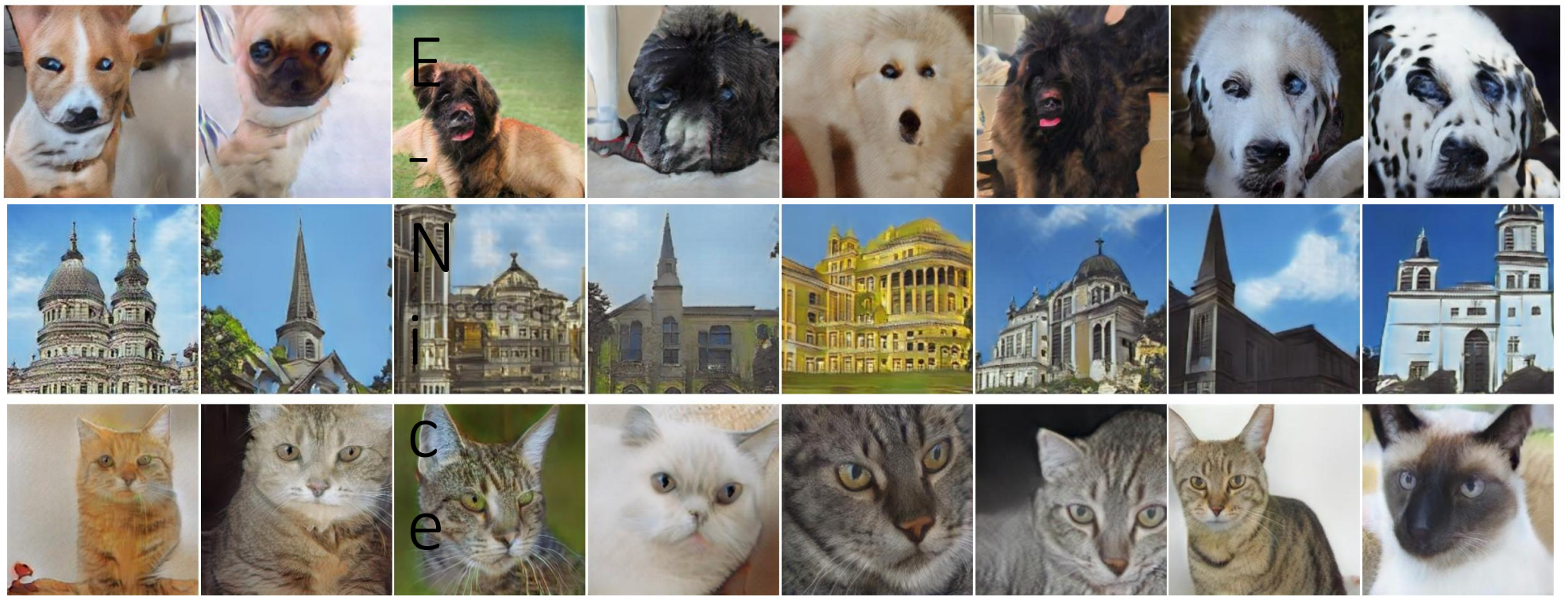


Man with glasses - man + woman = woman with glasses

[Radford et al. (2015): UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS]

Image Generation:

Samples from the “generated distribution”



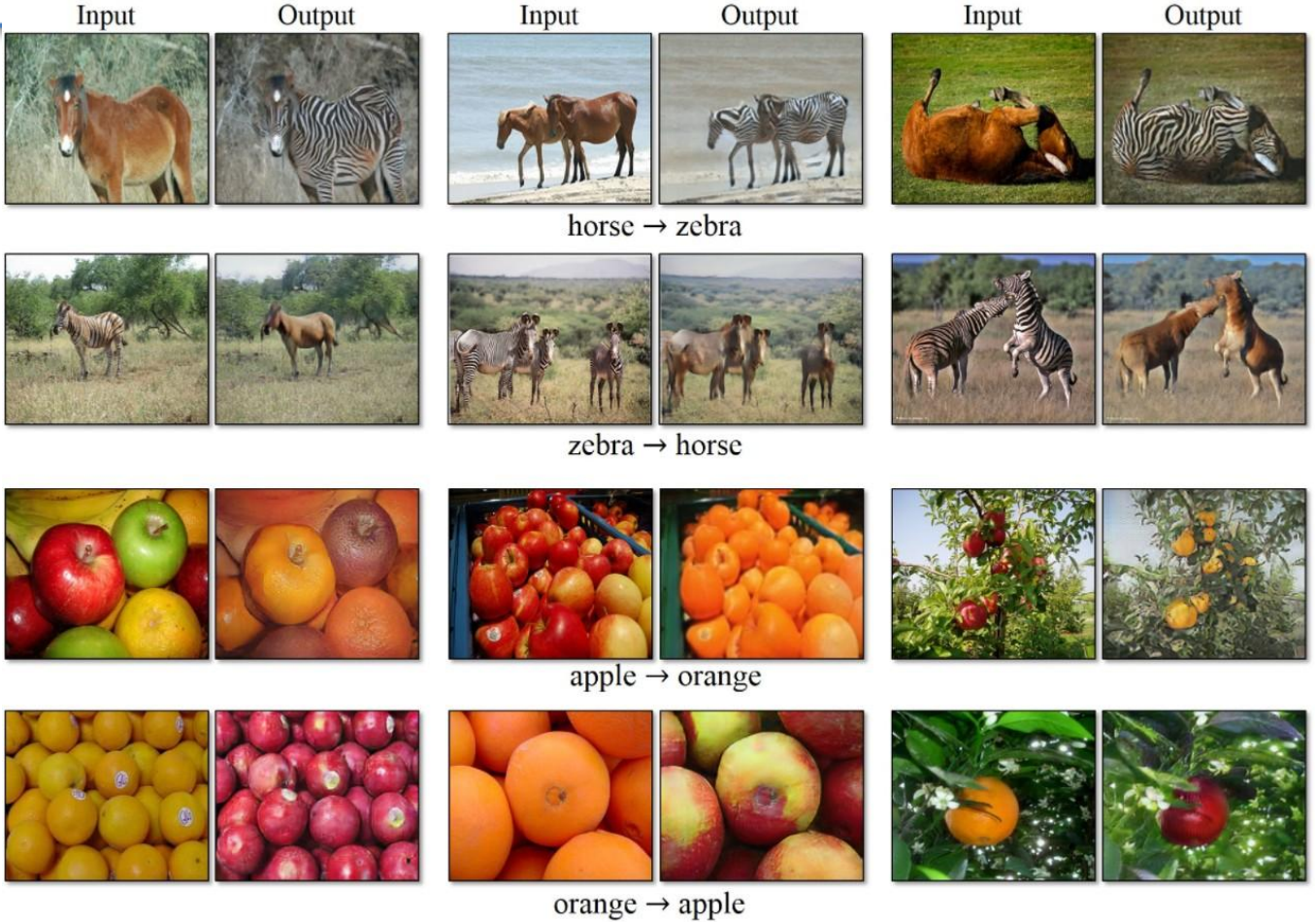
[Zhang et al. (2017): StackGAN++]

II.E - Nice results

- [Karras et al. (2018): A Style-Based Generator Architecture for Generative Adversarial Networks]

https://youtu.be/kSLJriaOumA

II.E - Nice results



[Zhu, Park et al. (2017): Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks]

II.E - Nice results

Goal: Convert horses to zebras on images, and vice-versa.

Data? **Architecture?** **Cost?**

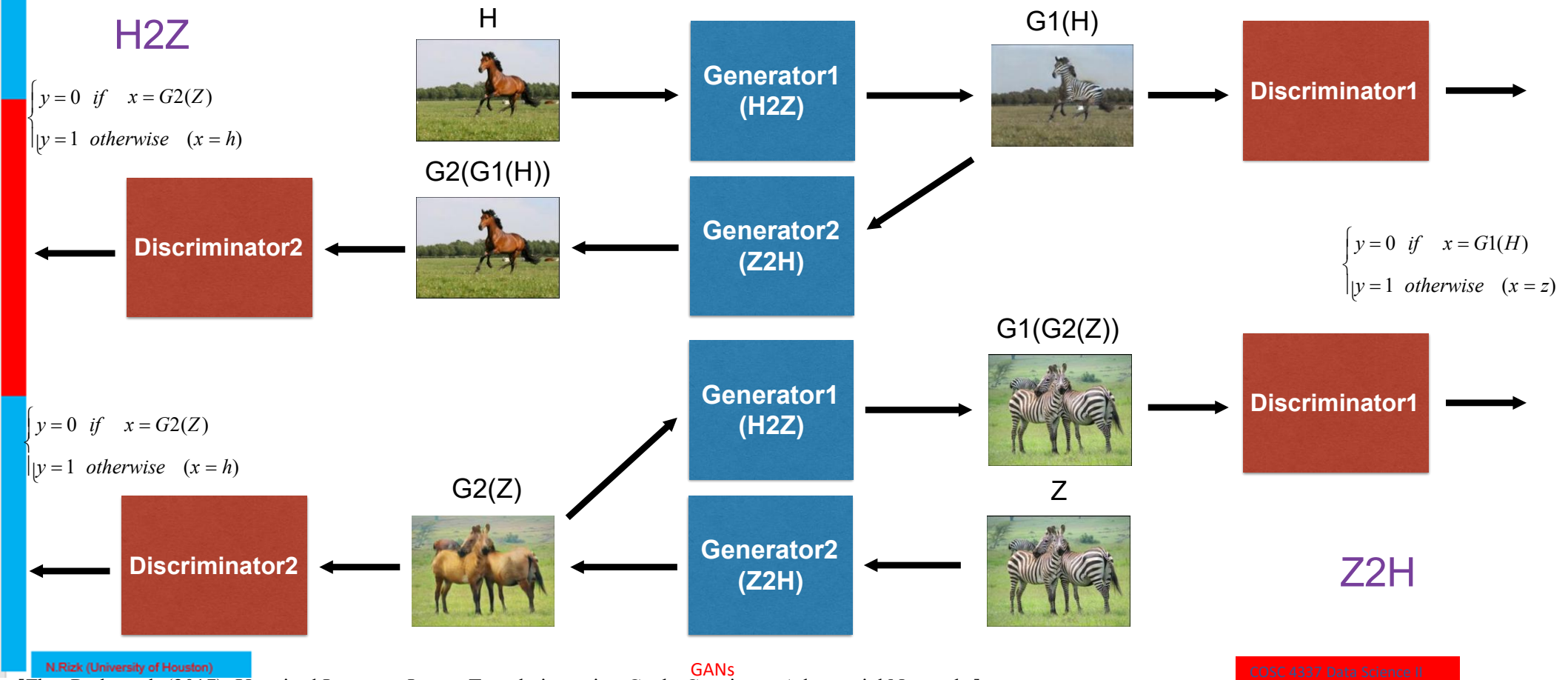
Unpaired images

Horse images Zebra images

[Zhu, Park et al. (2017): Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks]

II.E - Nice results

Architecture?



[Zhu, Park et al. (2017): Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks]

II.E - Nice results

Loss to minimize?

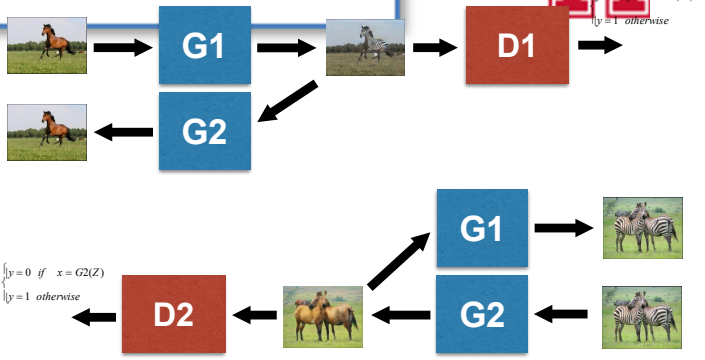
$$J^{(D1)} = -\frac{1}{m_{real}}\sum_{i=1}^{m_{real}}\log(D1(z^{(i)})) - \frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(1 - D1(G1(H^{(i)})))$$

$$J^{(G1)} = -\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D1(G1(H^{(i)})))$$

$$J^{(D2)} = -\frac{1}{m_{real}}\sum_{i=1}^{m_{real}}\log(D2(h^{(i)})) - \frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(1 - D2(G2(Z^{(i)})))$$

$$J^{(G2)} = -\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\log(D2(G2(Z^{(i)})))$$

$$J^{cycle} = -\frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\| G2(G1(H^{(i)}) - H^{(i)} \|_1 - \frac{1}{m_{gen}}\sum_{i=1}^{m_{gen}}\| G1(G2(Z^{(i)}) - Z^{(i)} \|_1$$



$$J = J^{(D1)} + J^{(G1)} + J^{(D2)} + J^{(G2)} + \lambda J^{cycle}$$

II.E - Nice results

CycleGANs:

Face2ramen

+ Face detection

[Shu Naritomi et al.: Face2Ramen]
[Takuya Tako; Face2Ramen using CycleGAN]
[Zhu, Park et al. (2017): Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks]

II.E - Nice results

Super-resolution

Human Portrait Super Resolution Using GANs

Yujie Shu
[redacted]

Figure 1: Input LR 32x32, SRPGGAN 8x Output 256x256, and Original HR 256x256

[Ledig et al. (2016): Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network]

Summary

Component	Description
Generator	Simple 2-layer MLP: noise → 28×28 image
Discriminator	2-layer binary classifier: real/fake
Dataset	Fashion-MNIST (28×28 grayscale)
Training time	≈ 2–3 minutes on CPU
Result	After ~1500–2000 epochs, you'll see the generator producing rough clothing-like textures

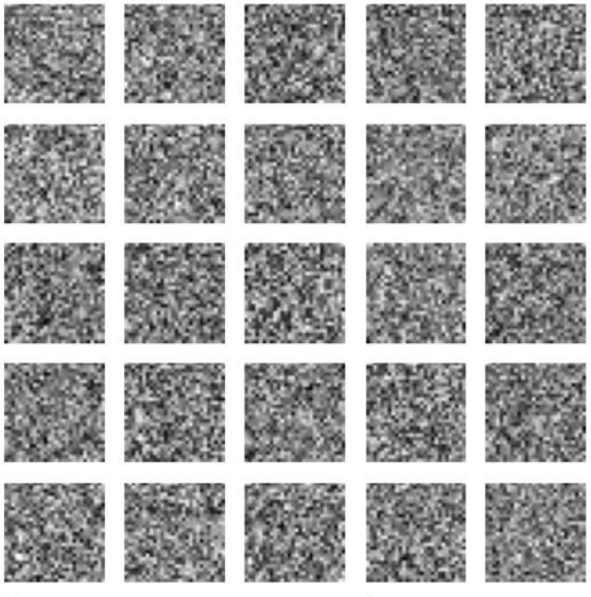
```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU, Reshape, Flatten
from tensorflow.keras.optimizers import Adam

# --- 1. Load and preprocess data ---
(X_train, _), (_, _) = fashion_mnist.load_data()
X_train = (X_train.astype(np.float32) - 127.5) / 127.5  # scale to [-1, 1]
X_train = X_train.reshape(-1, 784)

# --- 2. Generator ---
def make_generator():
    model = Sequential([
        Dense(128, input_dim=100),
        LeakyReLU(0.2),
        Dense(784, activation='tanh'),
        Reshape((28, 28))
    ])
    return model

# --- 3. Discriminator ---
def make_discriminator():
    model = Sequential([
        # Remove the Flatten layer as input is already flattened
        Dense(128, input_dim=784), # Add input_dim here
        LeakyReLU(0.2),
        Dense(1, activation='sigmoid')
    ])
    return model

# --- 4. Compile both models ---
optimizer = Adam(0.0002, 0.5)
generator = make_generator()
discriminator = make_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])
```

```python
# --- 5. Combined model (Generator + frozen Discriminator) ---
discriminator.trainable = False
gan = Sequential([generator, Flatten(), discriminator]) # Add Flatten layer here
gan.compile(loss='binary_crossentropy', optimizer=optimizer)

# --- 6. Training loop ---
def train(epochs=2000, batch_size=64, save_interval=500):
    half = batch_size // 2

    for epoch in range(epochs):
        # --- Train Discriminator ---
        idx = np.random.randint(0, X_train.shape[0], half)
        real_imgs = X_train[idx]

        noise = np.random.normal(0, 1, (half, 100))
        fake_imgs = generator.predict(noise)
        fake_imgs = fake_imgs.reshape(half, 784)  # Flatten fake images

        d_loss_real = discriminator.train_on_batch(real_imgs, np.ones((half, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs, np.zeros((half, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # --- Train Generator ---
        noise = np.random.normal(0, 1, (batch_size, 100))
        valid = np.ones((batch_size, 1))
        g_loss = gan.train_on_batch(noise, valid)

        # --- Print progress ---
        if epoch % 100 == 0:
            print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc: {100*d_loss[1]:.2f}%] [G loss: {g_loss:.4f}]")

        # --- Save generated samples ---
        if epoch % save_interval == 0:
            show_generated(epoch)
```

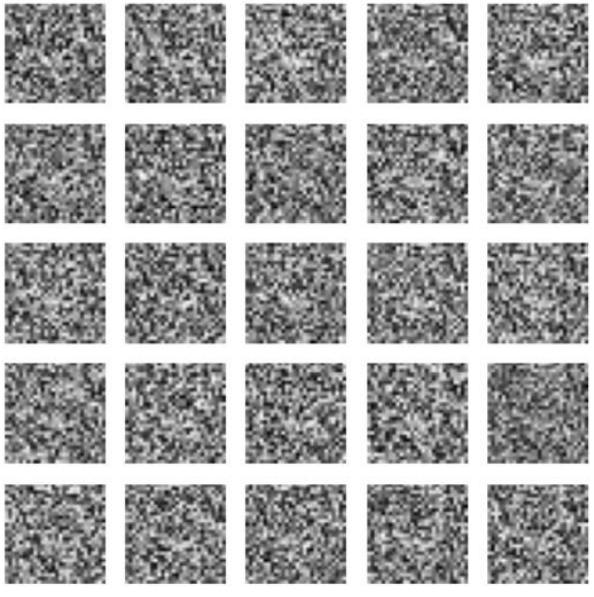
```python
# --- 7. Helper to visualize outputs ---
def show_generated(epoch):
    noise = np.random.normal(0, 1, (25, 100))
    # Predict method of the generator outputs images with shape (28, 28), flatten them for the discriminator
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5

    fig, axs = plt.subplots(5, 5, figsize=(5,5))
    count = 0
    for i in range(5):
        for j in range(5):
            axs[i,j].imshow(gen_imgs[count], cmap='gray')
            axs[i,j].axis('off')
            count += 1
    plt.suptitle(f"Generated Samples - Epoch {epoch}")
    plt.show()
    plt.close()

# --- 8. Train it ---
train(epochs=2000, batch_size=64, save_interval=500)
```

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step

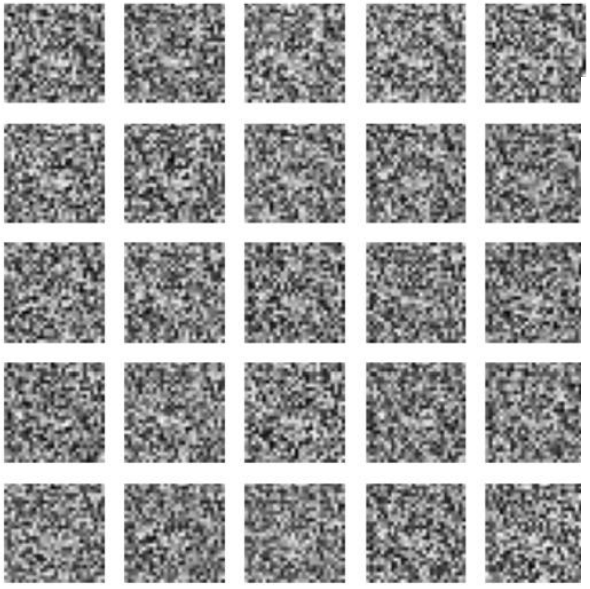
Generated Samples - Epoch 0



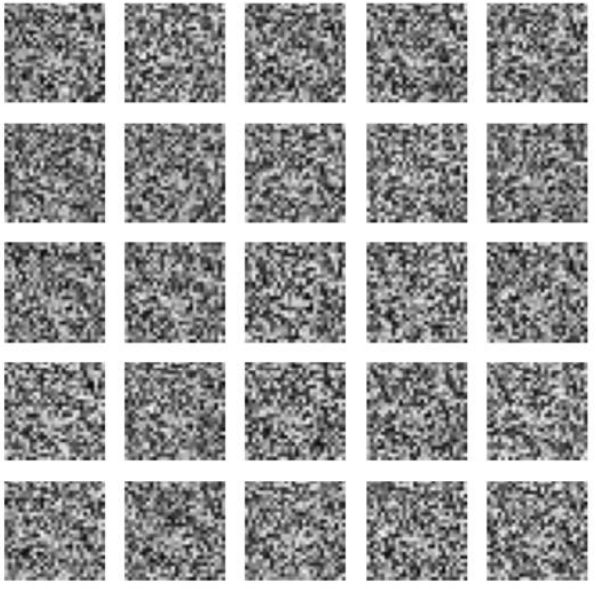
Generated Samples - Epoch 500



Generated Samples - Epoch 1000



Generated Samples - Epoch 1500



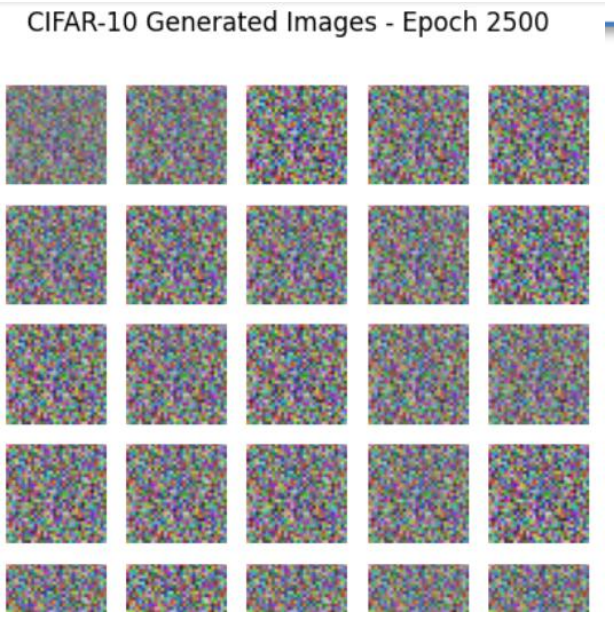
Key Notes

Component	Description
Dataset	CIFAR-10 (32×32 RGB images of animals, vehicles, etc.)
Generator	Fully connected network → reshaped to (32, 32, 3)
Discriminator	2 hidden layers, binary classifier
Training time	~5–8 min on CPU / <2 min on GPU
Result	Generates coarse colored blobs that gradually resemble CIFAR objects after ~2000 epochs

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU, Reshape, Flatten
from tensorflow.keras.optimizers import Adam

# --- 1. Load and preprocess data ---
(X_train, _), (_, _) = cifar10.load_data()
X_train = (X_train.astype(np.float32) - 127.5) / 127.5  # scale to [-1, 1]
# X_train = X_train.reshape(-1, 32*32*3) # Remove flattening here

latent_dim = 100  # random noise vector size

# --- 2. Generator ---
def make_generator():
    model = Sequential([
        Dense(256, input_dim=latent_dim),
        LeakyReLU(0.2),
        Dense(512),
        LeakyReLU(0.2),
        Dense(1024),
        LeakyReLU(0.2),
        Dense(32*32*3, activation='tanh'),
        Reshape((32, 32, 3))
    ])
    return model

# --- 3. Discriminator ---
def make_discriminator():
    model = Sequential([
        Flatten(input_shape=(32, 32, 3)),
        Dense(512),
        LeakyReLU(0.2),
        Dense(256),
        LeakyReLU(0.2),
        Dense(1, activation='sigmoid')
    ])
    return model
```

```python
# --- 4. Compile models ---
optimizer = Adam(0.0002, 0.5)
generator = make_generator()
discriminator = make_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# --- 5. Combined model ---
discriminator.trainable = False
gan = Sequential([generator, discriminator])
gan.compile(loss='binary_crossentropy', optimizer=optimizer)

# --- 6. Training loop ---
def train(epochs=3000, batch_size=64, save_interval=500):
    half = batch_size // 2

    for epoch in range(epochs):
        # --- Train Discriminator ---
        idx = np.random.randint(0, X_train.shape[0], half)
        real_imgs = X_train[idx]

        noise = np.random.normal(0, 1, (half, latent_dim))
        fake_imgs = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_imgs, np.ones((half, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs, np.zeros((half, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # --- Train Generator ---
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        valid = np.ones((batch_size, 1))
        g_loss = gan.train_on_batch(noise, valid)

        # --- Print progress ---
        if epoch % 100 == 0:
            print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc: {100*d_loss[1]:.2f}%] [G loss: {g_loss:.4f}]")

        # --- Save generated samples ---
        if epoch % save_interval == 0:
            show_generated(epoch)
```

```python
# --- 7. Helper to visualize generated images ---
def show_generated(epoch):
    noise = np.random.normal(0, 1, (25, latent_dim))
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5  # scale back to [0,1]

    fig, axs = plt.subplots(5, 5, figsize=(5,5))
    count = 0
    for i in range(5):
        for j in range(5):
            axs[i, j].imshow(gen_imgs[count])
            axs[i, j].axis('off')
            count += 1
    plt.suptitle(f"CIFAR-10 Generated Images - Epoch {epoch}")
    plt.show()
    plt.close()

# --- 8. Train the GAN ---
train(epochs=3000, batch_size=64, save_interval=500)
```



Generator: Learns to map random noise → fake clothing images.

Discriminator: Learns to distinguish real Fashion-MNIST samples from generated ones.

Training: Alternates between training the discriminator on real and fake images, and training the generator to fool the discriminator.

Visualization: Every 1000 epochs, you'll see samples of generated "clothing" evolve from random noise to realistic shapes.

Conditional GAN (cGAN) using **Fashion-MNIST**, where the generator learns **to produce specific clothing items** (like shirts, shoes, or bags) based on a **class label**.

This example conditions both the **generator** and **discriminator** on a class label (0–9) from Fashion-MNIST.

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Input, Dense, LeakyReLU, Dropout, Flatten, Reshape
from tensorflow.keras.optimizers import Adam

# --- 1. Load and preprocess Fashion-MNIST data ---
(X_train, _), (_, _) = fashion_mnist.load_data()
X_train = X_train / 127.5 - 1.0   # Normalize to [-1, 1]
X_train = X_train.reshape(-1, 784)  # Flatten 28x28 -> 784

latent_dim = 100  # noise vector size

# --- 2. Build the Generator ---
def build_generator():
    model = Sequential([
        Dense(256, input_dim=latent_dim),
        LeakyReLU(alpha=0.2),
        Dense(512),
        LeakyReLU(alpha=0.2),
        Dense(1024),
        LeakyReLU(alpha=0.2),
        Dense(784, activation='tanh'),
        Reshape((28, 28))
    ])
    return model

# --- 3. Build the Discriminator ---
def build_discriminator():
    model = Sequential([
        Dense(512, input_dim=784), # Input shape is already flattened
        LeakyReLU(alpha=0.2),
        Dropout(0.3),
        Dense(256),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),
        Dense(1, activation='sigmoid')
    ])
    return model
```

```
# --- 4. Compile the Discriminator ---
optimizer = Adam(0.0002, 0.5)
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

# --- 5. Combine Generator + Discriminator ---
generator = build_generator()
discriminator.trainable = False  # freeze D during G training

# Define GAN as a Model for explicit input/output flow
z = Input(shape=(latent_dim,))
img = generator(z)
# Flatten the generated image before feeding to discriminator
flat_img = Flatten()(img)
validity = discriminator(flat_img)

gan = Model(z, validity)
gan.compile(loss='binary_crossentropy', optimizer=optimizer)
```

```python
# --- 6. Training loop ---
def train(epochs=10000, batch_size=128, save_interval=1000):
    half_batch = batch_size // 2

    for epoch in range(epochs):
        # --- Train discriminator ---
        idx = np.random.randint(0, X_train.shape[0], half_batch)
        real_imgs = X_train[idx]
        noise = np.random.normal(0, 1, (half_batch, latent_dim))
        fake_imgs = generator.predict(noise)
        fake_imgs = fake_imgs.reshape(half_batch, 784)  # Flatten fake images

        d_loss_real = discriminator.train_on_batch(real_imgs, np.ones((half_batch, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs, np.zeros((half_batch, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # --- Train generator ---
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        valid_y = np.ones((batch_size, 1))
        g_loss = gan.train_on_batch(noise, valid_y)

        # --- Print progress ---
        if epoch % 200 == 0:
            print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc.: {100*d_loss[1]:.2f}%] [G loss: {g_loss:.4f}]")

        # --- Save sample images ---
        if epoch % save_interval == 0:
            save_imgs(epoch)
```

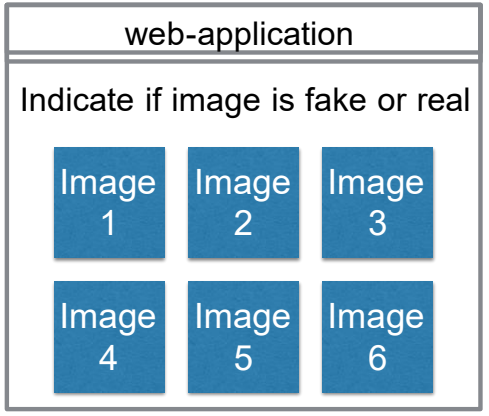
```python
# --- 7. Helper function to display generated images ---
def save_imgs(epoch):
    r, c = 5, 5
    noise = np.random.normal(0, 1, (r * c, latent_dim))
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5  # scale back to [0, 1]

    fig, axs = plt.subplots(r, c, figsize=(5,5))
    count = 0
    for i in range(r):
        for j in range(c):
            axs[i,j].imshow(gen_imgs[count], cmap='gray')
            axs[i,j].axis('off')
            count += 1
    plt.suptitle(f"Fashion-MNIST Generated Images at Epoch {epoch}")
    plt.tight_layout()
    plt.show()
    plt.close()

# --- 8. Train the GAN ---
train(epochs=5000, batch_size=64, save_interval=1000)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ub
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

II.E - Evaluating GANs

How to evaluate GANs?

Human annotators



Inception Score (IS)

$$IS(G) = \exp\left(\mathrm{E}_{x \sim p_g} \left[KL(p(y \mid x) \,||\, p(y)) \right] \right)$$

Measure of image quality

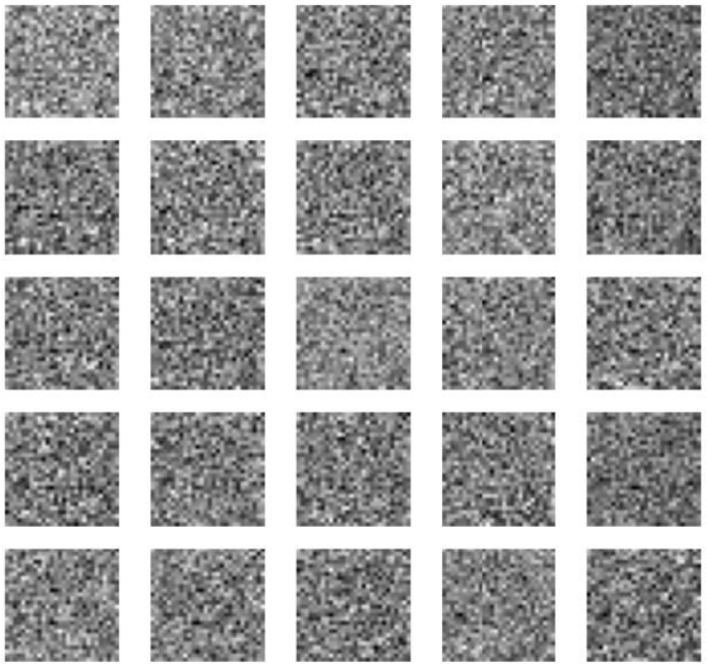
Measure of image diversity

$$IS(G) = \exp\left(\frac{1}{m_b} \sum_{i=1}^{m_b} \sum_{j=1}^{n_y} \hat{y}_j^{(i)} \left(\log \hat{y}_j^{(i)} - \log\left(\frac{1}{m_b} \sum_{k=1}^{m_b} \hat{y}_j^{(k)} \right) \right) \right)$$

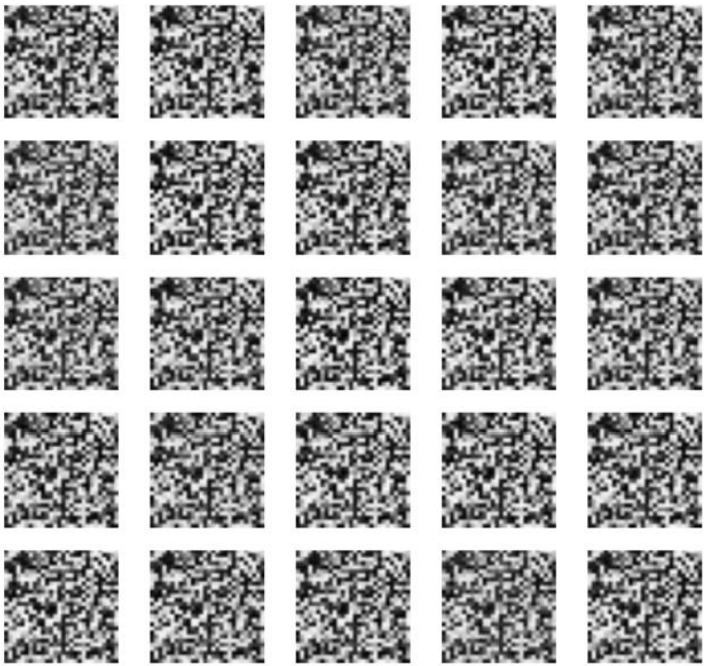
Also: Fréchet Inception Distance (FID)

[Lucic, Kurach et al. (2018): Are GANs Created Equal? A Large-Scale Study]
[Salimans et al. (2016): Improved Techniques for Training GANs]

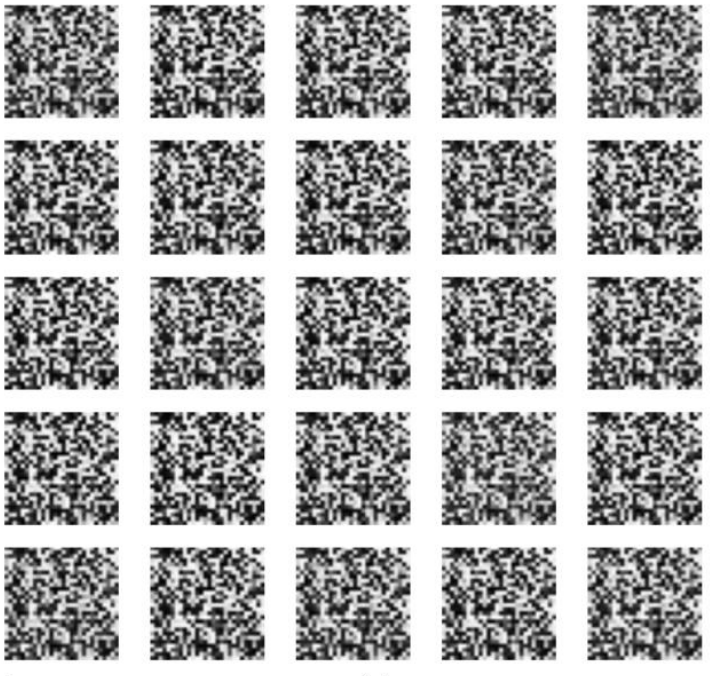
Fashion-MNIST Generated Images at Epoch 0



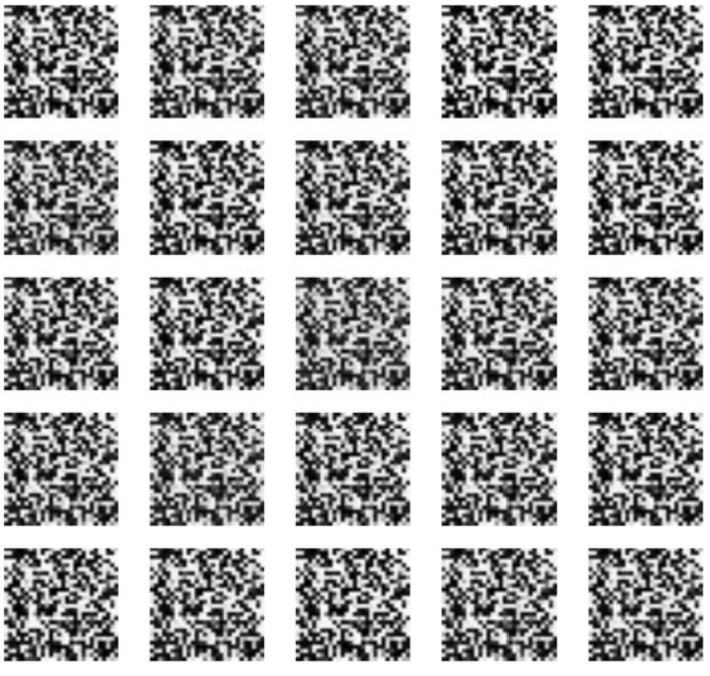
Fashion-MNIST Generated Images at Epoch 1000



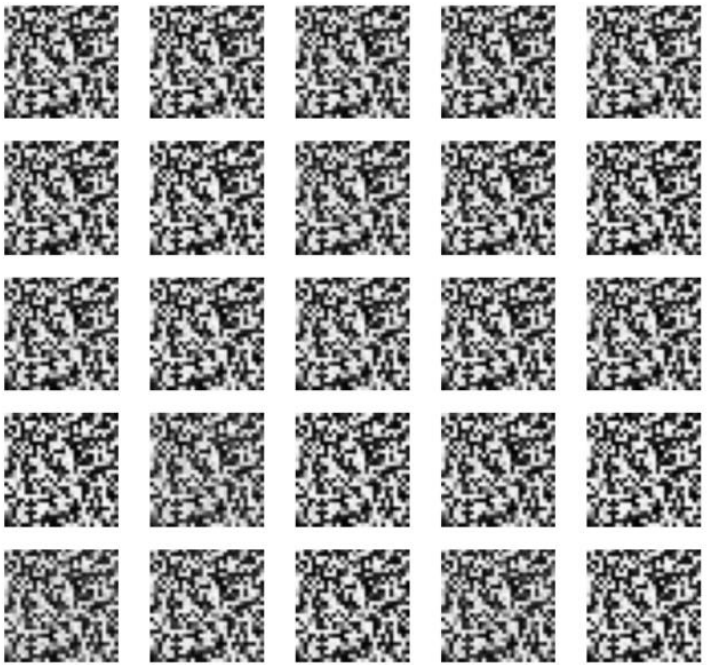
Fashion-MNIST Generated Images at Epoch 2000



Fashion-MNIST Generated Images at Epoch 4000



Fashion-MNIST Generated Images at Epoch 5000



DCGAN (Deep Convolutional GAN) using **CIFAR-10** — this is the **standard architecture** that produces *realistic-looking* small color images (dogs, airplanes, cars, etc.) using convolutional layers.

Summary

Component	Description
Generator	Upsamples from 4×4 to 32×32 using Conv2DTranspose layers
Discriminator	CNN that classifies real vs fake
Dataset	CIFAR-10 (32×32 RGB)
Training time	~5–10 min on GPU, ~20–30 min on CPU
Output	Colorful 32×32 images resembling CIFAR-10 objects after ~4000–5000 epochs

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Reshape, Conv2DTranspose, Conv2D, LeakyReLU, Flatten,
from tensorflow.keras.optimizers import Adam

# --- 1. Load and preprocess data ---
(X_train, _), (_, _) = cifar10.load_data()
X_train = (X_train.astype(np.float32) - 127.5) / 127.5  # normalize to [-1, 1]
latent_dim = 100

# --- 2. Build the Generator (ConvTranspose network) ---
def build_generator():
    model = Sequential()
    model.add(Dense(256 * 4 * 4, input_dim=latent_dim))
    model.add(LeakyReLU(0.2))
    model.add(Reshape((4, 4, 256)))  # start small

    model.add(Conv2DTranspose(128, kernel_size=4, strides=2, padding="same"))
    model.add(LeakyReLU(0.2))

    model.add(Conv2DTranspose(128, kernel_size=4, strides=2, padding="same"))
    model.add(LeakyReLU(0.2))

    model.add(Conv2DTranspose(64, kernel_size=4, strides=2, padding="same"))
    model.add(LeakyReLU(0.2))

    model.add(Conv2D(3, kernel_size=3, padding="same", activation='tanh'))
    return model
```

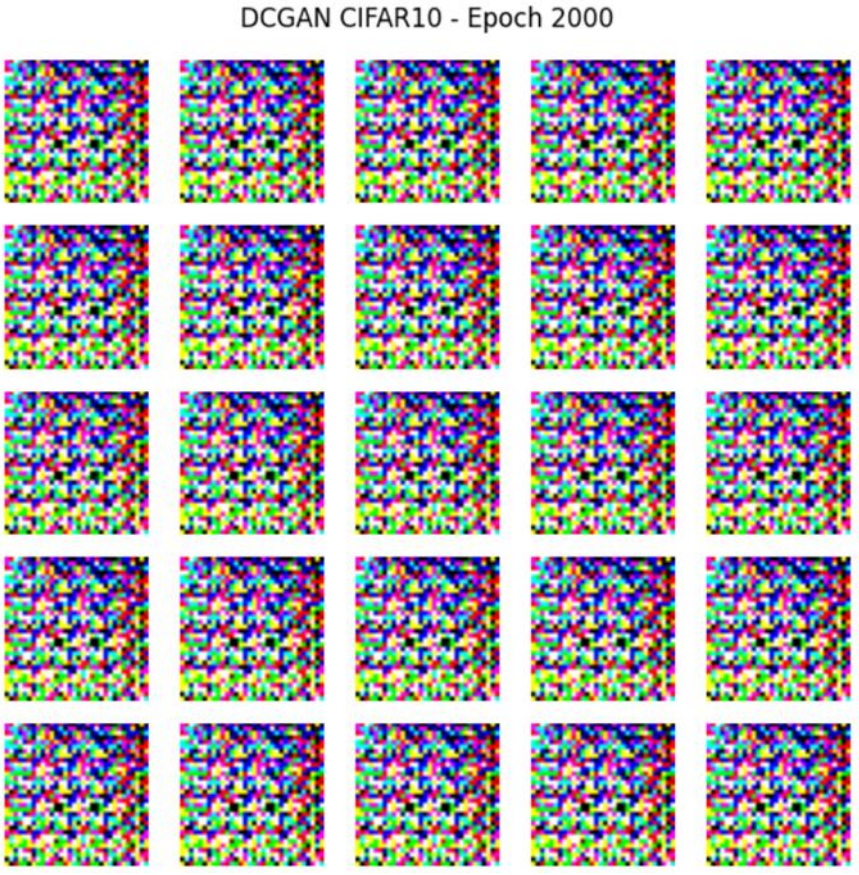
```python
# --- 3. Build the Discriminator (Conv network) ---
def build_discriminator():
    model = Sequential()
    model.add(Conv2D(64, kernel_size=3, strides=2, padding="same", input_shape=(32, 32, 3)))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))

    model.add(Conv2D(128, kernel_size=3, strides=2, padding="same"))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))

    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))
    return model

# --- 4. Compile models ---
optimizer = Adam(0.0002, 0.5)
discriminator = build_discriminator()
discriminator.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy'])

generator = build_generator()
discriminator.trainable = False

from tensorflow.keras import Model, Input
z = Input(shape=(latent_dim,))
img = generator(z)
validity = discriminator(img)
dcgan = Model(z, validity)
dcgan.compile(loss='binary_crossentropy', optimizer=optimizer)
```

```python
# --- 5. Training loop ---
def train(epochs=5000, batch_size=64, save_interval=1000):
    half = batch_size // 2

    for epoch in range(epochs):
        # --- Train Discriminator ---
        idx = np.random.randint(0, X_train.shape[0], half)
        real_imgs = X_train[idx]

        noise = np.random.normal(0, 1, (half, latent_dim))
        fake_imgs = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_imgs, np.ones((half, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs, np.zeros((half, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # --- Train Generator ---
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        valid_y = np.ones((batch_size, 1))
        g_loss = dcgan.train_on_batch(noise, valid_y)

        # --- Display progress ---
        if epoch % 100 == 0:
            print(f"{epoch} [D loss: {d_loss[0]:.4f}, acc: {100*d_loss[1]:.2f}%] [G loss: {g_loss:.4f}]")

        # --- Generate samples every few epochs ---
        if epoch % save_interval == 0:
            show_generated(epoch)
```

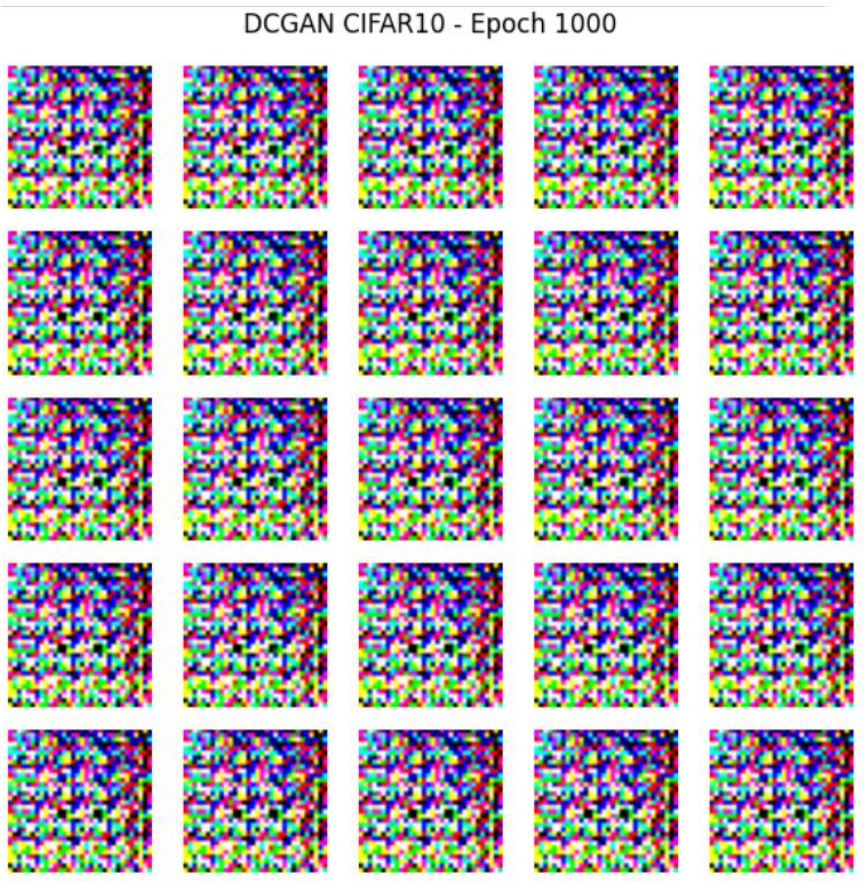
```python
# --- 6. Visualization helper ---
def show_generated(epoch):
    r, c = 5, 5
    noise = np.random.normal(0, 1, (r * c, latent_dim))
    gen_imgs = generator.predict(noise)
    gen_imgs = 0.5 * gen_imgs + 0.5  # scale back to [0, 1]

    fig, axs = plt.subplots(r, c, figsize=(6,6))
    count = 0
    for i in range(r):
        for j in range(c):
            axs[i, j].imshow(gen_imgs[count])
            axs[i, j].axis('off')
            count += 1
    plt.suptitle(f"DCGAN CIFAR10 - Epoch {epoch}")
    plt.tight_layout()
    plt.show()
    plt.close()

# --- 7. Train DCGAN ---
train(epochs=5000, batch_size=64, save_interval=1000)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 154ms/step
2600 [D loss: 2.1418, acc: 9.67%] [G loss: 0.0361]
```

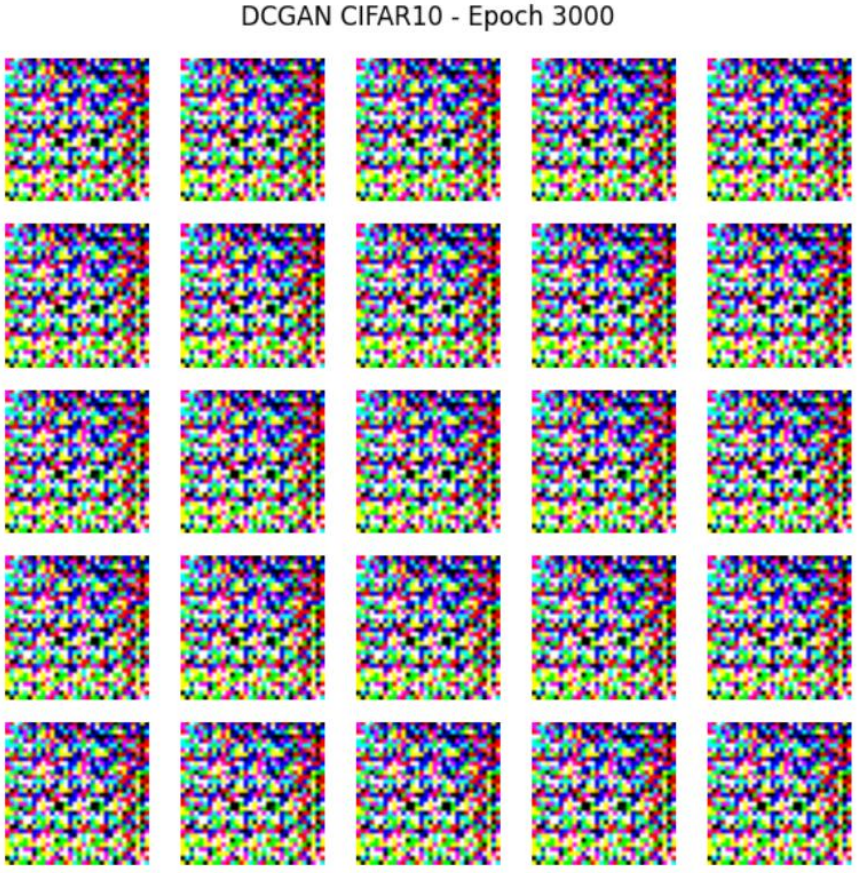
After 44 minutes



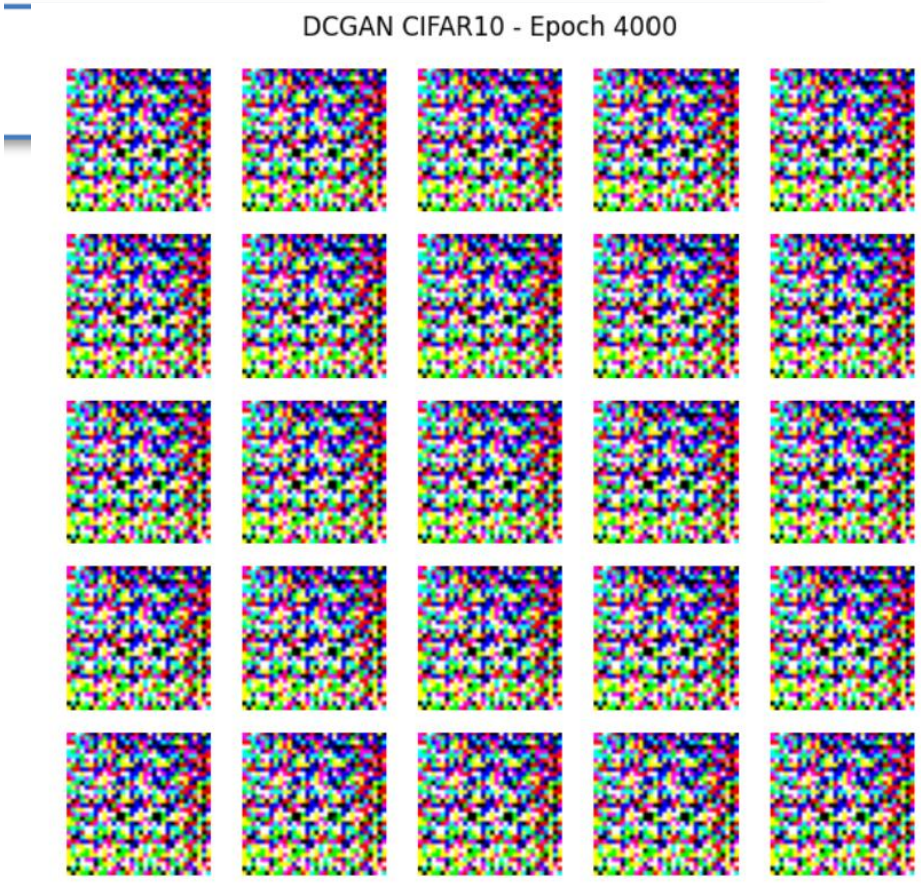
After 37 minutes



After 20 minutes



After 54 minutes



After 98 minutes

DCGANs are the first GAN architecture that made image generation “realistic enough” to be useful — they bridge the gap between toy GANs and modern image synthesis models.

Benefit	Explanation
Higher-quality images	Learns local structure with convolutions
Training stability	Less gradient instability than MLP GANs
Fewer parameters	Shared convolutional weights
Transferable features	Discriminator learns reusable representations
Scalable to large datasets	Works with higher-resolution inputs
Smooth latent space	Enables realistic interpolation between images

Benefits of DCGANs

Better Image Quality

•Convolutional layers learn **spatial hierarchies** — edges, textures, shapes — instead of treating pixels as independent.

•This helps the generator create *realistic, coherent images* rather than blurry blobs.

•The discriminator also becomes far better at distinguishing real vs. fake images.

Example:

A fully connected GAN trained on CIFAR-10 may produce colorful noise, while a DCGAN learns patterns resembling airplanes, dogs, or cars.

Parameter Efficiency

•CNNs use **shared weights** and **local receptive fields**, so they need **fewer parameters** to capture structure in 2D images.

•This reduces memory footprint and makes training more stable and efficient.

Stability in Training

•DCGANs typically converge faster and more reliably than dense GANs.

•The architectural guidelines (e.g., using LeakyReLU, BatchNorm, strided convolutions) help prevent **mode collapse** and **gradient explosion/vanishing**.

Learned Feature Representations

•The discriminator's convolutional layers learn useful **unsupervised visual features**.

•These learned features can be reused (fine-tuned) for **image classification** or **transfer learning** — even though no labels were used.

Example:

Features from a DCGAN's discriminator can help train a classifier with fewer labeled samples.

Scalability

•DCGANs scale naturally to **larger and more complex datasets** (e.g., CelebA, LSUN, ImageNet subsets).

•They are the **foundation** for most advanced GANs:

- Conditional GANs (cGANs)
- StyleGAN
- CycleGAN
- Pix2Pix

Smooth Latent Space

•Because of convolutional upsampling, interpolating between two random noise vectors produces **smooth transitions** in the generated images (e.g., a shoe slowly morphing into a shirt).

•This shows the generator learned a continuous and meaningful data manifold.